Group members:

Student Names and IDs:

Maimuna Aminu Suleiman 210208950

SIDI LAMINE ABDOURAHMANE 220208889

Zakariyya Zakariyya Suleiman 230208857

TrustVerify: File Integrity and Authenticity Verification Tool

Introduction

In modern information systems, ensuring the integrity and authenticity of files is critical, especially in cybersecurity contexts. This project presents the implementation of a Python-based Command Line Interface (CLI) tool called *TrustVerify*. The tool allows a sender to generate hashes of files, create a manifest, and digitally sign it. A receiver can then verify both the integrity of the files and the authenticity of the sender.

The system combines two important cryptographic concepts: hashing (SHA-256) and digital signatures (RSA). While hashing ensures that files have not been modified, digital signatures ensure that the data originates from a trusted source.

Part 1: Hashing and File Integrity

The first part of the project focuses on detecting file modifications using SHA-256 hashing. A hash function takes an input file and produces a fixed-length output (hash value). Even a small change in the file results in a completely different hash.

The system performs the following operations:

- Generates a SHA-256 hash for each file in a directory
- Stores the results in a `metadata.json` file (manifest)
- Compares current file hashes with stored hashes to detect:
  - Modified files
  - Missing files
  - Newly added files

This ensures **data integrity**, meaning any unauthorized changes to the files can be detected.

Why Hashing Alone is Not Enough

Although hashing is effective for detecting changes, it does not provide any information about the origin of the data. Anyone can generate a hash for a file, including an attacker.

For example:

- An attacker can modify a file
- Recalculate the hash
- Update the `metadata.json`

In this case, the integrity check would still pass, even though the data is no longer trustworthy.

Therefore, hashing alone cannot guarantee **authenticity** or **identity**. It only ensures that data has not changed relative to the stored hash, but it does not prove who created that hash.

Part 2: Digital Signatures (RSA)

To address the limitations of hashing, the second part of the project introduces digital signatures using RSA cryptography.

The system performs the following:

1. Generates a public/private key pair
2. Computes the SHA-256 hash of the `metadata.json`
3. Signs the hash using the sender's private key
4. Allows a receiver to verify the signature using the sender's public key

This ensures that:

- The manifest has not been altered
- The sender's identity is verified

How Public/Private Keys Ensure Non-Repudiation

RSA uses two keys:

- **Private Key** (kept secret by the sender)
- **Public Key** (shared with others)

When the sender signs the hash using their private key:

- Only the corresponding public key can verify it

This provides **non-repudiation**, meaning:

- The sender cannot deny creating the signature
- Only the owner of the private key could have generated it

If the file or the manifest is modified after signing:

- The hash changes
- Signature verification fails

This guarantees both **authenticity** and **integrity**.

System Workflow

Sender Side:

1. Generate file hashes and create `metadata.json`
2. Generate RSA key pair
3. Sign the manifest using the private key

Receiver Side:

1. Verify file integrity using `metadata.json`
2. Verify the digital signature using the public key

Conclusion

The TrustVerify tool demonstrates how combining hashing and digital signatures creates a secure system for verifying both file integrity and authenticity. Hashing alone is insufficient because it cannot prove the origin of data. However, when combined with RSA digital signatures, the system ensures that data has not been altered and that it originates from a trusted sender.

This project highlights the importance of cryptographic techniques in securing digital communication and protecting data from tampering and impersonation.